Closure evidence — heading-regex widening + corpus repair

Captured: 2026-09-01T18:36:11Z HEAD: ec25327

Identity-audit unlocated count (review MINOR-5)

	count
BEFORE (period-only regex, pre-repair)	55
AFTER (widened+tightened, post-repair)	57

The count ROSE by 2. This is an honest regression in that one number, not a silent one, and it is explained: 14 previously-invisible blocks now parse into the SSoT, and the audit can only LOCATE a block that declares a `**TMX-ID:**` line. Blocks with no such line assert no ownership and are skipped by design (the audit's own false-positive guard) — they are counted, never hidden.

file	blocks with no `**TMX-ID:**`	total blocks
Issues.md	3	8
Fixed.md	60	85

Sentinel identities (heading never parsed)

Measured against `git show HEAD~1:docs/workable_items.db` vs the current DB.

	BEFORE	AFTER
rows with `code_ordinal <= 0`	17	3 (of 94 rows)
rows with `category = 'Z'`	4	0

The 14 rows that healed (TMX-057..066, TMX-072..075) were the TRUE sentinels — every one of them now carries a real `<CAT><ORD>`. The 3 survivors are NOT sentinels: they are TMX-038 / TMX-039 / TMX-041, the genuine `### A0.` / `### B0` / `### C0` blocks at `Fixed.md:2585 / 2652 / 2718`, whose ordinal legitimately IS zero.

CORRECTION (§11.4.6): an earlier revision of this file published "7 → 3" for this row. That figure reproduces under no definition of "sentinel" the current DB supports (category-Z-only = 4; category-Z AND ordinal-0 = 4; `code_ordinal <= 0` = 17; true sentinels = 14) and it contradicted this document's own adjacent "14 newly-visible blocks" figure. The table above is the re-measured, reproducible replacement.

Why the DB holds 94 rows while `sync` parses 92 blocks (measured, reproducible)

	count
DB rows	94
parsed blocks	92
DISTINCT block identities `(location, CAT, ORD)`	91

94 = 91 distinct identities + 2 rows that SHARE an identity with a live successor (TMX-001 shares Fixed `B3` with TMX-054; TMX-071 shares Fixed `A52` with TMX-062) + 1 row whose claimed identity has no block at all (TMX-050 at Fixed `F1`). 92 = 91 distinct + 1 DUPLICATE markdown heading — `### A52.` occurs twice in `Fixed.md`, at line 2890 (TMX-062) and line 3027 (TMX-071).

All three extra rows carry status `Obsolete (→ Fixed.md)`, which is precisely the case `validate_identity.go`'s Obsolete guard skips (§11.4.90 — a superseded record may legitimately share its successor's block), so `validate` correctly reports 0 findings.

RETRACTION (§11.4.6): an earlier account of this gap attributed it to the two headings the tightened regex refuses (`### TMX-051 — …`, `### NEZHA-INSTALL-… —`). That mechanism is WRONG and is withdrawn — measured, NO DB row corresponds to either heading, so they contribute ZERO rows. The error was reading the id literal in `### TMX-051 —` as the row whose `atm_id` is TMX-051; that row is a different item entirely (Copy/paste mouse ownership, `A43`). That is the §11.4.201(9) field-identity class — an id literal matched as a row key — the same shape as the discarded 62-row instrument below. The

conclusions were unaffected (gap = 2, no live collision, `validate` 0 findings) but they were reached via the wrong rows, and the arithmetic above is the reproducible replacement.

Corpus duplication

Cross-tracker duplicates BEFORE: 5 (G1..G4 in both trackers; A50 claimed by two items). AFTER: 0 — enforced at the sync seam, which now refuses on the union of `**TMX-ID:**` and heading-identity across every parsed block in both files.

Mutation ledger (§1.1) — every guard pinned, each mutation verified to land before running

#	Mutation	Test that FAILs
M-C	revert `headingRE` to the loose pre-tightening form	`TestHeadingRE_AcceptedAndRefused`
M-D	desync `blockCodeRE` from `headingRE`	`TestHeadingRE_AndBlockCodeRE_AgreeOnAcceptance`
M-A	restrict the duplicate guard to same-code pairs	`…RefusesSameIDUnderDifferentBlockCodes`
M-B	restrict the duplicate guard to cross-file pairs	`…RefusesSameIDTwiceWithinOneFile`
R1	disable the byHash key	`…RefusesIdLessDuplicateAcrossTrackers`
R5	swap identity-resolution precedence	`…HeadingHashWinsAndNoSpuriousRebindIsRecorded`
R17		`…IdentityRebindIsRecordedInHistory`

#	Mutation	Test that FAILs
	suppress the rebind counter, keep the history row	
R23	strip the prior identity from the history `Reason`	`…IdentityRebindIsRecordedInHistory`
—	persist `document_sources` before the guard	`…RefusalLeavesDocumentSourcesUntouched`
—	remove the rebind's history recording	`…IdentityRebindIsRecordedInHistory`
—	blank `RebindDetails`	`…IdentityRebindIsRecordedInHistory`
R2	disable the id-reservation loop	`TestAllocatorReservesIDLiteralInsideANonParseableBlock`
R3	drop the atm-branch heading-hash refresh	`TestUpsertItem_ChangedHeadingHashUpdatesByATMID`
R4	weaken `headingRE`'s title group to `(.*)$`	`TestHeadingRE_AcceptedAndRefused` + `…AgreeOnAcceptance`
R6	disable the greedy-bind guard	`TestParse_NonItemHeadingStillClosesMetadataWindow`
—	restore the `lookupByHeadingHash` error swallow	`…ReportsRealErrorsInsteadOfReportingAbsent`
—	drop `CodeOrdinal` from `itemContentEqual`	`…DriftedStoredOrdinalSelfHeals`
—	drop `HeadingHash` from `itemContentEqual`	`…StaleStoredHashSelfHeals`
—	drop BOTH identity fields	`…OrdinalOnlyRenumberIsAppliedAndRecorded`

Two of my own tests were BLIND on first write and were caught by running these mutations rather than trusting the green: the rebind test's assertions sat behind an inverted condition that skipped them every run, and the precedence test asserted a row outcome that is

identical under both orderings (`UpsertItem` re-resolves by heading-hash internally, so the sync-layer order changes only the audit trail). Both were rewritten to assert what the mutation actually changes.

Review trail (§11.4.134 iterate-to-GO)

Round 1 NO-GO — 1 BLOCKING (my corpus-removal loop over-deleted the `## H.` header and its §11.4.114 preamble; restored verbatim from `git show HEAD:Issues.md`), plus IMPORTANT + MINOR findings. Round 2 NO-GO — 3 IMPORTANT, 3 MINOR; it also VERIFIED the round-1 BLOCKING closed. Round 3 NO-GO — 1 IMPORTANT (the `itemContentEqual` identity gap, found in the seam I asked the reviewer to attack). Round 4 GO — zero findings, zero warnings. Delta review of the two filed tracker items GO. Delta review of `CONTINUATION.md` §3.37 + this file NO-GO on 3 MINOR documentation-accuracy findings (round attribution, the sentinel figures, and a superseded gitignored copy of this file) — all three remediated.

Round-numbering note (§11.4.6): no per-round review artifact was written to disk, so I could not reconstruct the round-1/2 boundary from local state. The numbering is the reviewer's record, now stated explicitly by it with a decisive argument: R1 and R5 ARE round 2's own findings (IMPORTANT-N1 and MINOR-N4), and a mutation that BECAME a round-2 finding cannot have survived round 1 — round 1's suite had never seen it. Only M-A survived round 1. Two revisions of this file got this wrong in opposite directions: the first folded round 1 into round 2 and mis-dated the BLOCKING; the second over-rotated and moved all three surviving mutations to round 1. This revision is the settled record.